All-Pairs Shortest Paths with Fox's Algorithm

Min-plus repeated squaring distributed with MPI

Faculty of Sciences · University of Porto

Antonio Lanção (up202205416)
David Sá (up202303580)
Henrique Teixeira (up202306640)

Parallel Computing 2026/27
25 October 2026

Abstract. We solve the all-pairs shortest path problem for directed weighted graphs by repeated squaring of the distance matrix under the min-plus semiring. Each product is distributed over a $Q \times Q$ process grid with Fox's algorithm, implemented in C with MPI. The communication of each step overlaps the computation of the previous one, and the iteration stops as soon as the distances no longer change. [TODO: one sentence with the headline result.]

1 Introduction

Given a directed graph with N nodes and non-negative edge weights, the all-pairs shortest path (APSP) problem asks for the length of the shortest path between every ordered pair of nodes. The graph is given as an $N \times N$ adjacency matrix W, where W_{ij} is the weight of the edge $i \to j$ and 0 off the diagonal means there is no edge.

This report describes a sequential baseline and a parallel implementation using Fox's algorithm over MPI, how both were validated, and how they perform on the LabCC cluster.

2 Algorithm

2.1 Min-plus product

The min-plus product of two $N \times N$ matrices replaces the sum by min and the product by $+$:

$$(A \otimes B)_{ij} = \min_{k=1}^{N} \big(A_{ik} + B_{kj}\big) \tag{1}$$

If $D^{(m)}$ holds the shortest distances using at most m edges, then $D^{(2m)} = D^{(m)} \otimes D^{(m)}$. Starting from $D^{(1)} = W$ (with ∞ for missing edges and 0 on the diagonal), $\lceil \log_2(N-1) \rceil$ squarings are enough, since a shortest path has at most $N-1$ edges. The total cost is $O\big(N^3 \log N\big)$, a $\log N$ factor above Floyd–Warshall, but every step is a matrix product, which parallelises well.

Both implementations also stop early: if a squaring leaves the matrix unchanged, every later squaring would too, so the current matrix is already the answer.

2.2 Fox's algorithm

The $P = Q^2$ processes form a $Q \times Q$ grid, and process (i, j) owns block (i, j) of A, B and C, each of size $N/Q \times N/Q$. Fox's algorithm computes $C = A \otimes B$ in Q steps:

```
for s = 0 .. Q-1:
    k = (i + s) mod Q
    broadcast A[i][k] along row i          # process (i, k) is the root
    C[i][j] = min(C[i][j], A[i][k] (x) B[k][j])
    shift B one row up (circularly) along column j
```

After step s, process (i, j) holds $B_{(i+s+1) \bmod Q, j}$, which is exactly the block it needs for the next value of k. Each process performs $Q \cdot (N/Q)^3 = N^3/P$ operations per product and, per step, receives one A block through a row broadcast and one B block from its neighbour.

3 Implementation

The code is split into a library without MPI (`lib/common`, `lib/seq`) and the distributed part (`lib/mpi`). A single `src/main.c` is compiled three times: as `seq`, as `fox` (with `-DUSE_MPI`) and as `fox_shared` (with `-DUSE_MPI -DFOX_SHARED`). The three programs share the input reader, the output writer and the min-plus kernel, and differ only in how the squarings are organised.

3.1 Data representation

Distances are stored as `uint32_t` in a row-major array. Missing edges are stored as $\mathrm{INF} = (2^{32} - 1)/2$, so the sum of two values that are each at most INF cannot overflow. The programs read and write a binary format: N as a 4-byte unsigned integer, followed by the N^2 entries in row-major order, with missing edges already stored as INF. The reader rejects a matrix whose diagonal is not 0. For the text format of the assignment, reading maps off-diagonal zeros to INF, and printing maps INF back to 0.

3.2 Sequential version

`seq` reads the matrix, runs `minplus_apsp` and writes the result. Only `minplus_apsp` is timed. It keeps two $N \times N$ buffers, the current distances D and the next product:

```
D = W
for covered = 1; covered + 1 < N; covered *= 2:
    Next = D (x) D            # minplus_multiply_into
    if Next == D: break       # early termination
    swap(D, Next)             # pointer swap, no copy
return D
```

`covered` is the largest number of edges that the paths in D can have. It doubles with each squaring, so the loop runs at most $\lceil \log_2(N-1) \rceil$ times. The product is written into the second buffer because the min-plus product cannot be computed in place: later entries of the product still need the old values of D.

3.2.1 Min-plus kernel

The min-plus kernel, `minplus_relax`, accumulates

$$C_{ij} \leftarrow \min\bigl(C_{ij}, A_{ik} + B_{kj}\bigr). \tag{2}$$

`minplus_multiply_into` fills C with INF and calls it once. Fox calls it once per step on the same C, which builds the full product. Three choices make it fast:

- **Loop order *i–k–j*.** The inner loop walks along a row of B and a row of C, both contiguous, so the compiler vectorises it with SIMD `min` instructions. With `-O3 -march=native` this is 8 lanes on AVX2.
- **Tiling.** The three loops are blocked in 128×128 tiles, so the parts of A, B and C in use stay in cache.
- **Skipping unreachable rows.** If $A_{ik} = \mathrm{INF}$, row k of B cannot improve row i of C and is skipped. Sparse inputs benefit most.

The `restrict` qualifiers tell the compiler that the three matrices do not alias. `floyd_warshall` is kept only as an independent reference for the unit tests.

3.3 Fox with scatter, broadcast and shift

`fox` follows the textbook algorithm in three stages. Rank 0 distributes the blocks once, every product moves B by point-to-point shifts and A by row broadcasts, and rank 0 collects the result at the end:

1 **Scatter**

Rank 0 reads W and packs its blocks in rank order. One `MPI_Scatter` gives process (i, j) its block $A = W_{ij}$, and B starts as a copy of A.

2 **Squarings — Q steps of `Irecv` B, `Isend` B, `Ibcast` A**

Each squaring is a Fox product of Q steps. In every step the process receives the next B from the process below, sends its current B to the process above, takes part in the row broadcast of the next A, and computes with the blocks it already has. An `MPI_Allreduce` then decides whether to stop; otherwise A takes the new block and B a copy of it, since the product rotates B.

3 **Gather**

One `MPI_Gather` collects the blocks in rank order, and rank 0 unpacks them into W.

In code (`mpi_apsp`):

```
rank 0: read W, pack its blocks in rank order
MPI_Scatter                  # process (i, j) receives A = W[i][j]
B = A
repeat at most ceil(log2(N - 1)) times:
    C = fox_multiply(A, B)
    changed = MPI_Allreduce(C != A, LOR)
    if not changed: break
    swap(A, C);  B = A       # B is rotated during the product, so it gets a copy
MPI_Gather                   # rank 0 unpacks the blocks into W
```

Each product (`fox_multiply`) runs the Q steps of Section 2.2, with the transfers for step $s + 1$ started before the computation of step s:

```
MPI_Bcast A[i][i] along row i                          # step 0
C = INF
for s = 0 .. Q-1:
    if s + 1 < Q:
        MPI_Irecv  B_next from (i + 1) mod Q along column j
        MPI_Isend  B_cur  to   (i - 1) mod Q along column j
        MPI_Ibcast A[i][(i + s + 1) mod Q] along row i  -> A_next
    C = min(C, A_cur (x) B_cur)                        # minplus_relax
    MPI_Waitall
    swap(B_cur, B_next);  A_cur = A_next
```

3.3.1 Process grid and distribution

Rank 0 reads the matrix and broadcasts N. Every rank then checks that P is a perfect square and that Q divides N. Otherwise rank 0 prints `ERROR: Invalid configuration!` and all ranks exit. The grid is a periodic two-dimensional Cartesian communicator (`MPI_Cart_create`, created without reordering so that grid rank 0 is the rank that read the input), and `MPI_Cart_sub` derives one communicator per row and one per column. The broadcasts of A use the row communicator, and the shifts of B use the column communicator.

Blocks are not contiguous in the global matrix, so rank 0 packs them in rank order into a buffer: block (i, j) becomes the $(iQ + j)$-th contiguous $N/Q \times N/Q$ piece. A single `MPI_Scatter` then gives

each process its block. At the end, `MPI_Gather` collects the blocks in the same order, and rank 0 unpacks them in place into the input matrix.

3.3.2 *Overlapping communication and computation*

The transfers of step $s+1$ run while step s computes:

- the next A block is broadcast with a non-blocking `MPI_Ibcast` into one of two alternating buffers, while the current one is in use;
- the B shift is an `MPI_Isend` of the current block to the process above and an `MPI_Irecv` of the next one from the process below, into a spare buffer. The two buffers are swapped after `MPI_Waitall`.

The computation only reads the buffers being sent, which MPI allows, so the step costs roughly $\max(t_{\text{comp}}, t_{\text{comm}})$ instead of their sum. Only the broadcast of step 0 is blocking, since nothing can be computed before it arrives. The process that is the broadcast root sends directly from its own A block, without a copy. All buffers are allocated once, in `mpi_apsp`, and reused across squarings. Each process holds six blocks (A, B, C, two A receive buffers and the spare B), i.e. $6(N/Q)^2 \cdot 4$ bytes.

3.3.3 *Early termination*

After each product, every process compares its new block with the previous one, and an `MPI_Allreduce` with `MPI_LOR` decides whether any block changed. The cost is one small collective per squaring, which can save several full products. `fox_shared` uses the same test.

3.4 Shared-copy variant with `MPI_Allgather`

`fox_shared` removes the communication from inside the product. Every process keeps a full copy M of the matrix, stored block by block in the same rank order as the scatter buffer of `fox`. Because every process knows where each block is, step s of the product reads A_{ik} and B_{kj}, with $k = (i + s) \bmod Q$, straight from memory (`fox_block` returns a view, not a copy), instead of receiving them by row broadcast and column shift. The communication moves to the end of each squaring:

1 Broadcast

Rank 0 reads W and packs its blocks in rank order into M. One `MPI_Bcast` gives every process the whole matrix.

2 Squarings — local product, then `MPI_Allgather`

Each process computes its block C in Q steps with no communication. After the `MPI_Allreduce` stop test, one `MPI_Allgather` puts every process's new block into every copy of M.

3 Unpack

Rank 0 already holds the whole result and unpacks M into W; no gather is needed.

In code:

```
rank 0: read W, pack its blocks in rank order -> M
MPI_Bcast M                   # every process gets the whole matrix
repeat at most ceil(log2(N - 1)) times:
    C = INF
    for s = 0 .. Q-1:
        k = (i + s) mod Q
        C = min(C, M[i][k] (x) M[k][j])
    changed = MPI_Allreduce(C != M[i][j], LOR)
    if not changed: break
    MPI_Allgather C -> M      # block of rank r lands at piece r
rank 0: unpack M into W
```

The block layout is what makes the all-gather work without packing: the piece that rank r contributes is exactly where block $(r/Q, r \bmod Q)$ belongs.

The trade-off is volume and memory, summarised in Table 1. Per squaring, `fox` delivers $2Q$ blocks to each process, $2N^2/Q$ values, in $2Q$ messages that overlap the computation. `fox_shared` delivers N^2 values to each process in one collective that does not overlap anything, $Q/2$ times more data, and each process holds $N^2 + (N/Q)^2$ values instead of $6(N/Q)^2$.

Table 1. Communication and memory of the two parallel versions, per process.

	`fox`	**`fox_shared`**
start	`MPI_Scatter`, N^2/P values	`MPI_Bcast`, N^2 values
each step	`MPI_Isend`/`MPI_Irecv` of B, `MPI_Ibcast` of A, overlapped	none
each squaring	$2Q$ blocks, $2N^2/Q$ values	`MPI_Allgather`, N^2 values, not overlapped
end	`MPI_Gather`, N^2/P values	none
memory	$6(N/Q)^2$ values	$N^2 + (N/Q)^2$ values

4 Validation

- **Unit tests** (`make test`): matrix allocation and copy, input parsing (including malformed input), and the min-plus kernel. The kernel is checked against the assignment's example, the identity matrix, all-INF matrices (overflow), split accumulation, and Floyd–Warshall on a random graph with $N = 150$, which is not a multiple of the tile size.
- **Regression** (`make check`): `seq` and `fox` are compared byte for byte with the reference outputs for $N \in \{6, 300, 600, 900, 1200, 3000\}$, with $P \in \{1, 4, 9, 16\}$. Configurations where Q does not divide N must be rejected with exactly the expected error message.

All tests pass.

5 Random inputs

The example inputs of the assignment stop at $N = 3000$ and are not powers of two, so a $Q \times Q$ grid only fits some of them. For the benchmarks we generate our own inputs with `tests/matgen`.

5.1 How `matgen` works

`matgen N [count] [seed]` writes `count` matrices of size $N \times N$, named `rand`k`_input`N`.bin`, in the binary format of Section 3.1. It seeds the C generator with `seed` (the current time if omitted) and fills the matrix in row-major order:

- the diagonal is 0;
- every other entry is an edge with probability 10%, otherwise INF;
- an edge gets a weight drawn uniformly from $[0, 2^{16}]$.

`rand()` may only produce 15 random bits, so each draw combines two calls into a 30-bit value before reducing it to $[0, 2^{16}]$. The weight bound keeps every distance far from INF: a shortest path has at most $N - 1$ edges, so for $N = 2^{14}$ it costs at most $(2^{14} - 1) \cdot 2^{16} \approx 1.07 \cdot 10^9$, below INF $\approx 2.15 \cdot 10^9$. `make gen` runs `matgen` for $N = 2^6, ..., 2^{14}$ with seed 42 and zips the files, so every machine benchmarks exactly the same matrices.

5.2 Why 90% of the entries are INF

The density decides what the input looks like as a graph, so we measured its effect before choosing it. Table 2 runs the sequential `minplus_apsp` on matrices generated like `matgen`'s, with seed 42, at several densities.

Table 2. Effect of edge density on the sequential algorithm: squarings performed (including the last, which only confirms convergence) out of $\lceil \log_2(N-1) \rceil$, pairs left unreachable, and time on a laptop (AMD Ryzen 7 7840HS).

N	edges	squarings	unreachable	time (ms)
64	100%	5 / 6	0%	0.2
64	10%	5 / 6	0%	0.1
64	1%	4 / 6	95.9%	0.0
256	100%	5 / 8	0%	8.9
256	10%	5 / 8	0%	8.9
256	1%	6 / 8	18.1%	6.2
2048	100%	6 / 11	0%	4996
2048	10%	6 / 11	0%	4572
2048	1%	6 / 11	0%	3636
2048	0.1%	6 / 11	33.1%	1477

Why 10% edges

- **Every pair is still connected.** At 10% no pair is unreachable for any $N \geq 64$, so the output is a full distance matrix. At 1% a graph with 64 nodes leaves 96% of the pairs unreachable, and its "result" is mostly INF.
- **The work is not made easier.** A 10% matrix needs as many squarings as a complete graph and, at $N = 2048$, takes 92% of its time. Sparser inputs finish faster only because the kernel skips INF entries in the first squarings, which would flatter the measurements.
- **Missing edges are exercised.** A complete graph never contains INF, so the overflow guard and the skip in `minplus_relax` would never run. With 90% INF the first squarings meet them everywhere.

Two consequences follow. The number of squarings barely depends on density, between 5 and 6 here, so early termination saves about half of the $\lceil \log_2(N-1) \rceil$ products on these inputs. And since the weights can be 0, `matgen` may produce zero-weight edges; the binary format keeps them as edges, unlike the text format, where 0 means no edge.

6 Performance evaluation

6.1 Methodology

[TODO: LabCC node specification: CPU, cores per node, memory, network, compiler and MPI versions, and how processes were mapped to nodes.]

Times are wall-clock times of the computation only (scatter, squarings and gather), measured on rank 0 with `MPI_Wtime`. The clock starts after an `MPI_Barrier` and stops once the gather returns. Input and output are excluded. Each configuration was run [TODO: R] times and the minimum is reported. The benchmark is driven by `bench/run_bench.sh`. Speedup and efficiency are computed relative to the sequential program:

$$S(P) = \frac{T_{\text{seq}}}{T(P)}, \quad E(P) = \frac{S(P)}{P} \tag{3}$$

6.2 Results

Table 3. Execution time in milliseconds.

N	**seq**	$P = 1$	$P = 4$	$P = 9$	$P = 16$	$P = 25$
300						
600						
900						
1200						
3000						

Table 4. Speedup relative to the sequential version.

N	$P = 1$	$P = 4$	$P = 9$	$P = 16$	$P = 25$
300					
600					
900					
1200					
3000					

6.3 Random matrices on LabCC

Figure 1 and Table 5 show `seq` and `fox` on the random matrices `rand0_input`N of Section 5, $N = 2^6, \ldots, 2^{14}$, run on LabCC with `bench/run_bench.sh` and a hostfile of 29 nodes with 8 slots each. Each configuration ran twice and the minimum is reported. Of the requested $P \in \{4, 9, 25, 36, 64, 81, 100, 144, 196\}$, only $P = 4$ ($Q = 2$) and $P = 64$ ($Q = 8$) have a Q that divides a power of two; the others were skipped. Processes fill the hostfile slot by slot, so $P = 4$ runs on a single node and $P = 64$ on eight full nodes.

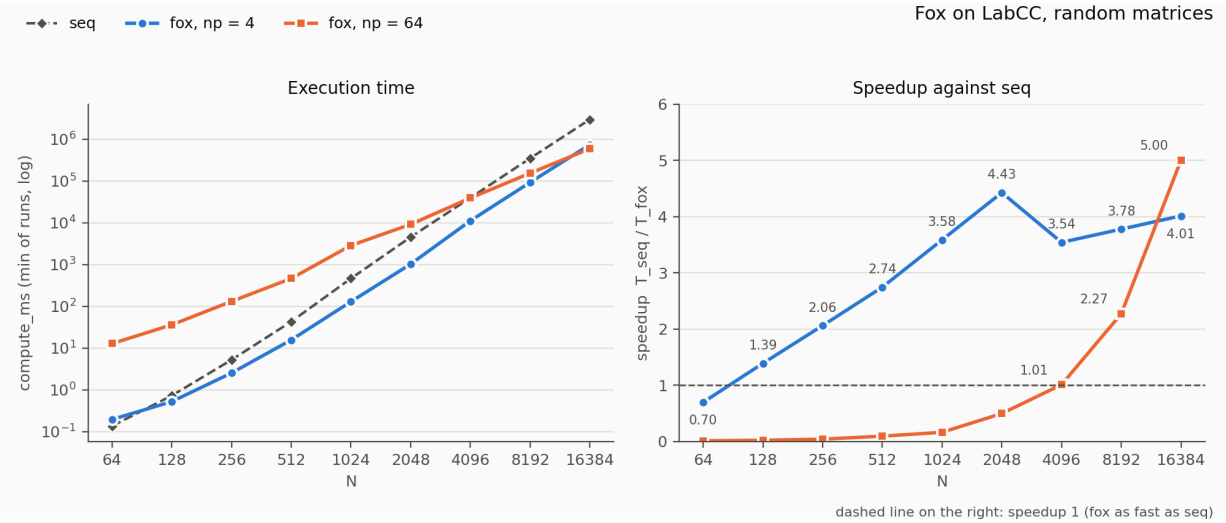


Figure 1. Left: minimum time per N on a log scale, `seq` dashed. Right: speedup against `seq`; below the dashed line, `fox` is slower than `seq`. Speedups under 0.5 are left unlabelled; they are in Table 5.

Table 5. Random matrices on LabCC: minimum of 2 runs in milliseconds, speedup S and efficiency E relative to `seq`.

		$P = 4$			$P = 64$		
N	seq	time	S	E	time	S	E
64	0.137	0.197	0.70	0.17	12.9	0.01	0.000
128	0.732	0.527	1.39	0.35	36.2	0.02	0.000
256	5.24	2.54	2.06	0.52	131.5	0.04	0.001
512	43.1	15.7	2.74	0.69	465.1	0.09	0.001
1024	464.6	129.6	3.58	0.90	2869	0.16	0.003
2048	4547	1027	4.43	1.11	9174	0.50	0.008
4096	39508	11157	3.54	0.89	39133	1.01	0.016
8192	349261	92488	3.78	0.94	153850	2.27	0.035
16384	2964332	738863	4.01	1.00	592689	5.00	0.078

With four processes on one node, `fox` is slower than `seq` only for $N = 64$, and from $N = 1024$ on its efficiency stays between 0.89 and 1.11. The values slightly above 1 ($N = 2048$ and $N = 16384$) come from each process working on an $N/2 \times N/2$ block, which uses the cache better than the full matrix in `seq`.

With 64 processes on eight nodes the picture is reversed. Up to $N = 2048$, `fox` is slower than `seq`, by up to two orders of magnitude for small N: each product has 8 steps, each with a row broadcast and a column shift that now cross the network, and with $N/8 \leq 256$ there is too little computation per step to hide them. It breaks even at $N = 4096$, and only at $N = 16384$ (blocks of 2048×2048) does it beat $P = 4$, with a speedup of 5.0 against 4.0. Even then the efficiency is 0.08, so a 16-fold increase in processes buys a 1.25-fold reduction in time. $P = 64$ only pays off for matrices far larger than those in the assignment.

The two runs of each configuration agree within 4% for $N \geq 2048$; for smaller N, where times are a few milliseconds, they differ by up to 32%.

6.4 Fox against the shared-copy variant

Figure 2 and Table 6 compare `fox` and `fox_shared` on random matrices with $N = 2^6, \ldots, 2^{10}$ (`matgen`, seed 42), measured with `bench/run_power2.sh` on a laptop with an AMD Ryzen 7 7840HS (8 cores,

16 hardware threads). Values are the median of 3 runs (6 for $P \in \{4, 16\}$). $P = 64$ oversubscribes the machine, four processes per hardware thread, so those points show contention, not scaling.

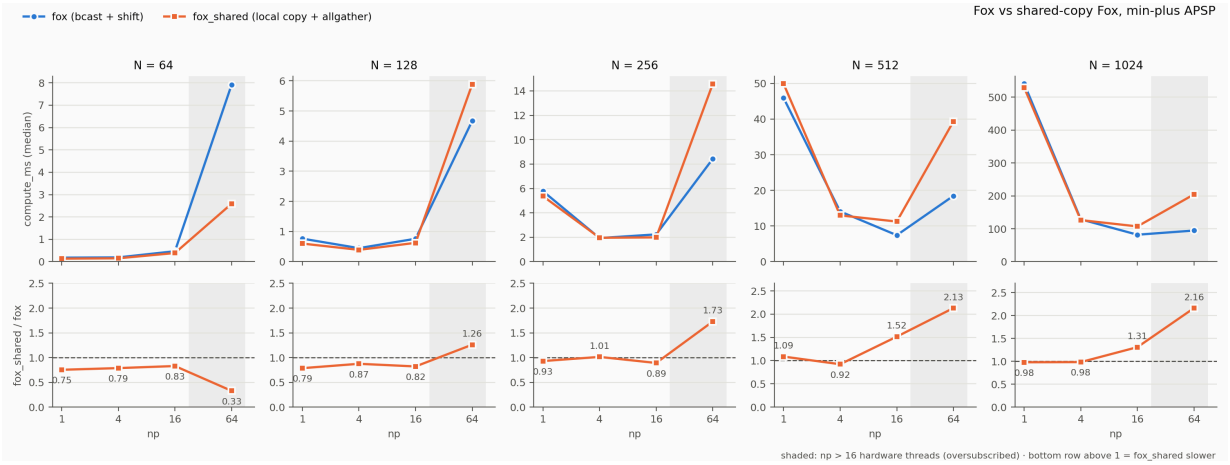


Figure 2. Top: median time per P, one panel per N, each with its own scale. Bottom: `fox_shared` time divided by `fox` time; above 1, `fox_shared` is slower. Shaded: P above the 16 hardware threads.

Table 6. Median execution time in milliseconds, `fox` against `fox_shared`.

	$P = 1$		$P = 4$		$P = 16$		$P = 64$	
N	fox	shared	fox	shared	fox	shared	fox	shared
64	0.18	0.13	0.19	0.15	0.47	0.39	7.92	2.59
128	0.76	0.60	0.45	0.39	0.76	0.62	4.67	5.88
256	5.78	5.37	1.93	1.96	2.23	1.99	8.44	14.6
512	46.0	50.0	14.1	13.0	7.42	11.3	18.5	39.4
1024	541.3	529.4	128.6	126.3	81.9	107.1	94.6	204.6

For $N \leq 256$, `fox_shared` is slightly faster (0.8–0.9 of the time of `fox` at $P \leq 16$). The blocks are tiny there, so the cost of `fox` is the latency of its $2Q$ messages per product rather than their size, and one all-gather per squaring is cheaper. As N grows the volume takes over: at $P = 16$, `fox_shared` takes 1.52 times as long as `fox` for $N = 512$ and 1.31 times for $N = 1024$, which is the $Q/2 = 2$ times larger transfer, not hidden behind computation. At $P = 4$ the two are within a few percent, since $Q/2 = 1$: both move the same amount of data. At $P = 1$ both run the same single local product.

6.5 Discussion

[TODO: Discuss Table 3 and Table 4: overhead of $P = 1$ against `seq`; scaling with P for each N; where communication starts to dominate (small N, large P); effect of using several nodes; effect of early termination.]

7 Conclusions

[TODO: Summary of results and possible improvements.]